

Weather Forecast Pipeline

Final Project Documentation

An Airflow-orchestrated ETL pipeline for Open-Mateo hourly forecast data

Developed by: Yasser Rezk Youssef

[LinkedIn](#)

[GitHub](#)

[Project Link](#)

Version 1.0 • June 30, 2026

1. Project Overview

The Weather Forecast Pipeline is an automated ETL system that fetches hourly weather forecast data from the Open-Meteo API, validates and reshapes it, compares it against existing records, and loads it into a PostgreSQL database for analysis and visualization. The pipeline is orchestrated with Apache Airflow 3.2 running under Celery Executor via Docker Compose, and ships with an interactive Dashboard to explore the collected data.

1.1 Architecture

```

Open-Meteo API
|
EXT01 (Extract) → src/api/weather_api.py
|
TRA01 (Transform) → src/preprocessing/clean_data.py
|
PRELOAD01 (Compare) → src/etl/compare.py
|
LOAD01 (Load) → src/etl/load.py → weather-Postgres
  
```

Each stagehands its output to the next via Airflow XCom, with the DataFrame serialized to JSON between tasks. The pipeline runs four sequential PythonOperator tasks within a single DAG (dag01), scheduled hourly.

1.2 Service Topology

Service	Role
postgres	Airflow metadata database
weather-postgres	Dedicated database for collected weather data
redis	Celery message broker
airflow-apiserver	Airflow 3 web UI and REST API
airflow-scheduler	Schedules and triggers DAG runs
airflow-dag-processor	Parses DAG files
airflow-triggerer	Handles deferred/async tasks
airflow-worker	Executes task instances (Celery worker)
flower (optional)	Celery monitoring UI, enabled via --profile flower

1.3 Technology Stack

Component	Technology
Orchestration	Apache Airflow 3.2.2 (CeleryExecutor)
Containerization	Docker Compose
Data source	Open-Meteo Forecast API
Data processing	pandas, SQLAlchemy
Database	PostgreSQL
Visualization	Dash + Plotly (dash-bootstrap-components)

2. API Endpoints

The pipeline integrates with a single external data source – the Open-Meteo Forecast API – accessed through one internal extraction function. There is no exposed REST API in this project; the term “endpoint” below refers to the external API the pipeline consumes, and the internal entry points used by the DAG.

2.1 External Endpoint – Open-Meteo Forecast

Property	Value
Base URL	https://api.open-meteo.com/v1/forecast
Method	GET
Auth	None required (public API)
Timeout	30 seconds (configured in <code>weather_api.py</code>)
Response format	JSON

Query Parameters Used

Parameter	Example Value	Purpose
latitude / longitude	27, 30 (Cairo)	Forecast location
hourly	temperature_2m, relative_humidity_2m, rain, ...	Comma-separated list of hourly variables to return
models	best_match	Weather model selection
time zone	Africa/Cairo	Timezone used for returned timestamps

The full request URL is configured via the `WEATHER_API_URL` environment variable in `env`, requesting the following hourly fields: `temperature_2m`, `relative_humidity_2m`, `rain`, `uv_index`, `is_day`, `sunshine_duration`, `dew_point_2m`, `apparent_temperature`, `precipitation_probability`,

surface_pressure, cloud_cover, visibility, wind_speed_10m, wind_direction_10m, and wind_gusts_10m.

Expected Response Shape

```
{
  "latitude": 27.0,
  "longitude": 30.0,
  "timezone": "Africa/Cairo",
  "elevation": 123.0,
  "hourly": {
    "time": ["2026-06-30T00:00", "2026-06-30T01:00", ...],
    "temperature_2m": [29.4, 28.1, ...],
    "relative_humidity_2m": [41, 44, ...],
    "rain": [0.0, 0.0, ...],
    "uv_index": [0.0, 0.0, ...],
    "is_day": [0, 0, ...],
    "sunshine_duration": [0.0, 0.0, ...],
    "...": "additional hourly arrays"
  }
}
```

2.2 Internal Entry Points

These are the Python functions each DAG task invokes; they form the de facto internal “API” of the pipeline.

Function	Module	Description
<code>get_weather()</code>	<code>src/api/weather_api.py</code>	Issues the GET request to Open-Meteo and returns the parsed JSON payload. Raises on timeout, HTTP error, or missing ‘hourly’ key.
<code>clean_weather(data)</code>	<code>src/preprocessing/clean_data.py</code>	Validates required hourly columns, flattens the payload into a DataFrame, attaches location metadata, casts types, and drops rows with null key values.
<code>compare_with_db(df_new, engine=None)</code>	<code>src/etl/compare.py</code>	Differs a transformed batch against existing rows in <code>weather_forecasts</code> for the same location/time window; classifies rows as new, existing, changed, or stale.
<code>load_data(df)</code>	<code>src/etl/load.py</code>	Persists a DataFrame into the <code>weather_forecasts</code> table.
<code>get_engine()</code>	<code>src/database/connection.py</code>	Builds and validates a SQLAlchemy engine from <code>DB_*</code> environment variables.

DAG Task Sequence (dag01)

Task ID	Callable	Action
EXT01	extract()	Calls <code>get_weather()</code> and pushes the raw JSON to XCom
TRA01	transform()	Pulls raw JSON, runs <code>clean_weather()</code> , pushes cleaned DataFrame (as JSON) to XCom
PRELOAD01	pre_load()	Pulls cleaned data, runs <code>compare_with_db()</code> , logs a diff report, pushes the normalized <code>df_new</code> to XCom
LOAD01	load()	Pulls <code>df_new</code> and calls <code>load_data()</code> to write it to PostgreSQL

3. Database Schema Design

Weather data is stored in a single denormalized table, `weather_forecasts`, inside the weather database. Location metadata (latitude, longitude, timezone, elevation) is repeated on every row rather than normalized into a separate locations table – a deliberate single-table design that simplifies querying and loading at the cost of some redundancy.

3.1 Table: `weather_forecasts`

Column	Type	Notes
<code>forecast_id</code>	BIGSERIAL PK	Surrogate primary key
<code>latitude</code>	NUMERIC(8,5)	Required
<code>longitude</code>	NUMERIC(8,5)	Required
<code>timezone</code>	VARCHAR(50)	Required
<code>elevation</code>	NUMERIC(5,1)	Optional, supplied by Open-Meteo
<code>forecast_time</code>	TIMESTAMP WITH TIME ZONE	Required; hourly timestamp
<code>temperature_2m</code>	NUMERIC(4,1)	°C
<code>apparent_temperature</code>	NUMERIC(4,1)	“Feels like” temperature
<code>dew_point_2m</code>	NUMERIC(4,1)	
<code>relative_humidity_2m</code>	SMALLINT	0–100%
<code>rain</code>	NUMERIC(5,2)	mm
<code>precipitation_probability</code>	SMALLINT	0–100%
<code>surface_pressure</code>	NUMERIC(6,1)	hPa
<code>cloud_cover</code>	SMALLINT	0–100%

Column	Type	Notes
visibility	NUMERIC(7,1)	meters
uv_index	NUMERIC(3,1)	
wind_speed_10m	NUMERIC(5,1)	km/h or m/s
wind_direction_10m	SMALLINT	0–360 degrees
wind_gusts_10m	NUMERIC(5,1)	
sunshine_duration	NUMERIC(6,1)	Seconds
is_day	BOOLEAN	Converted from API's 1/0
model_used	VARCHAR(50)	Default 'best_match'
fetches_at	TIMESTAMP WITH TIME ZONE	Default CURRENT_TIMESTAMP

3.2 Constraints & Indexes

- `unique_lat_lon_time` – UNIQUE constraint on (latitude, longitude, forecast_time), preventing duplicate forecasts for the same place and hour.
- `idx_weather_time` – index on forecast_time DESC for fast recent-data queries.
- `idx_weather_geo_time` – composite index on (latitude, longitude, forecast_time DESC) for location-scoped range queries.

3.3 Design Notes

- Single-table design: chosen for simplicity over a normalized locations/forecasts split; acceptable while the pipeline tracks one location, but will duplicate metadata if multiple locations are added later.
- Schema initialization is idempotent (CREATE TABLE IF NOT EXISTS, CREATE INDEX IF NOT EXISTS) and is also auto-applied by `compare.py` if the table is missing at run time.

4. Usage Examples

4.1 First-Time Setup

1. Generate secrets and paste them into .env:

```
python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
python -c "import secrets; print(secrets.token_hex(32))"
```

2. On Linux/Mac, set the container user ID:

```
echo "AIRFLOW_UID=$(id -u)" >> .env
```

3. Start the weather database and initialize its schema:

```
docker compose up weather-postgres -d
docker exec -i weather_data_postgres psql -U admin -d weather < src/database/schema.sql
```

4. Start all services (Windows: run.bat):

```
docker compose up -d
```

5. Open the Airflow UI at <http://localhost:8080> (login airflow / airflow), then enable and trigger the “Weather Forecast Pipeline” DAG (dag01).

4.2 Fetching & Transforming Data Programmatically

```
from src.api.weather_api import get_weather
from src.preprocessing.clean_data import clean_weather

raw = get_weather()          # GET request to Open-Meteo
df = clean_weather(raw)      # validated, typed DataFrame
print(df.head())
```

4.3 Comparing a New Batch Against the Database

```
from src.etl.compare import compare_with_db, print_report

report = compare_with_db(df)
print_report(report)

# report.new_rows      -> rows not yet in the DB
# report.changed_rows  -> rows whose values drifted beyond tolerance (0.05)
# report.stale_rows    -> DB rows missing from the new batch
# report.column_drift  -> per-column drift statistics for changed rows
```

This module can also be run standalone for an ad-hoc comparison against live data:

```
python -m src.etl.compare
```

4.4 Loading Data

```
from src.etl.load import load_data

load_data(df)  # writes the DataFrame into weather_forecasts
```

4.5 Querying the Database Directly

```
SELECT forecast_time, temperature_2m, rain, uv_index
FROM weather_forecasts
```

```
WHERE latitude = 27.00000 AND longitude = 30.00000
AND forecast_time >= NOW() - INTERVAL '24 hours'
ORDER BY forecast_time;
```

4.6 Running the Dashboard

The Dash dashboard (`visualization/dashboard.py`) connects to the weather-postgres database on localhost and renders stat cards and charts (temperature, humidity, rain, UV index, sunshine) for each tracked location.

```
cd visualization
python dashboard.py
```

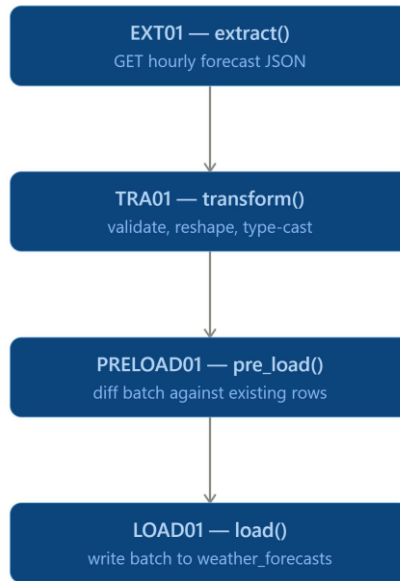
4.7 Operational Commands

Command	Purpose
<code>docker compose logs -f airflow-worker</code>	Tail worker logs
<code>docker compose logs -f airflow-scheduler</code>	Tail scheduler logs
<code>docker compose --profile flower up -d</code>	Start the Celery monitor at localhost:5555
<code>docker compose down -v</code>	Full reset – wipes both databases

5. Demo Walkthrough

Suggested flow for a live demonstration of the pipeline end to end:

- Bring up the stack with `docker compose up -d` and confirm all services report healthy in `docker compose ps`.
- Open the Airflow UI (localhost:8080) and show the dag01 graph view – EXT01 → TRA01 → PRELOAD01 → LOAD01.
- Trigger a manual DAG to run and follow task logs in real time, highlighting the compare-report output logged by PRELOAD01 (new / existing / changed / stale counts).
- Once LOAD01 succeeds, query `weather_forecasts` via `psql` or the `read_db.ipynb` notebook to show freshly loaded rows.
- Launch the Dash dashboard and walk through the stat cards and time-series charts for the configured location.
- Optionally enable the Flower profile to show Celery worker/task activity behind the scenes.



End of Document